ChatGPT said:

For an AV-grade system, I would not use the previous ESKF alone. I would use a **two-layer localization architecture**: a high-rate real-time ESKF for control/perception, backed by a fixed-lag nonlinear smoother that fuses IMU, wheel odometry, GNSS, LiDAR-map registration, and optionally visual landmarks.

The target output is not just a pose:

$$\boxed{\mathcal{X}(t) = \{T_{WE}(t), v_W(t), b_a(t), b_g(t), P(t), \text{quality}, \text{health}\}}$$

with a continuous pose history so LiDAR can query $T_{WE}(t_i)$ for every firing timestamp.

---

1. Production architecture

Plain text

```
            GNSS / RTK / INS
                   |
                   v
             GNSS frontend
                   |
                   +-----------------------------+
                                                 |
IMU ---> calibration ---> IMU propagation        |
 |                     200-1000 Hz               |
 |                         |                      |
 |                         v                      |
 |                     Real-time ESKF <-----------+
 |                         ^                      |
 |                         |                      |
Wheel/CAN ---> vehicle ----+                      |
               odometry                           |
                                                   |
LiDAR ---> deskew ---> registration ---> map ------+
 |                         |                      |
 |                         v                      |
```

A useful engineering principle is:

$$\boxed{\text{ESKF} = \text{fast local state}}$$

$$\boxed{\text{smoother} = \text{globally consistent recent trajectory}}$$

2. State

For keyframe k, use

$$X_k = \{R_k, p_k, v_k, b_{g,k}, b_{a,k}\}.$$

A practical error state is

$$\boxed{\delta x = [\delta p, \delta v, \delta\theta, \delta b_a, \delta b_g]^T}$$

with 15 dimensions.

Production systems frequently add states such as

$$\delta x_{\text{extended}} = [\delta p, \delta v, \delta\theta, \delta b_a, \delta b_g, \delta s_w, \delta t_{\text{sensor}}, \delta\theta_{\text{road}}, \ldots].$$

Here s_w might represent wheel-scale error.

I would generally keep factory-calibrated LiDAR/camera extrinsics fixed during normal operation rather than freely estimating them online unless the estimator has strong observability.

---

3. Continuous IMU dynamics

Use

$$\omega_m = \omega + b_g + n_g$$

and

$$a_m = R^T(a_W - g_W) + b_a + n_a.$$

Nominal dynamics:

$$\boxed{\dot{p} = v}$$

$$\boxed{\dot{v} = R(a_m - b_a) + g}$$

$$\boxed{\dot{R} = R[\omega_m - b_g]_\times}$$

$$\dot{b}_a = n_{ba}$$

$$\dot{b}_g = n_{bg}.$$

---

4. Don't use simple Euler integration

The educational implementation used essentially

$$R_{k+1} = R_k \backslash \mathrm{Exp}(\omega_k \Delta t).$$

For a production implementation I would at minimum use midpoint integration.

Given two consecutive IMU measurements:

$$\omega_k, \omega_{k+1}$$

and

$$a_k, a_{k+1},$$

calculate

$$\bar{\omega} = \frac{1}{2}[(\omega_k - b_g) + (\omega_{k+1} - b_g)].$$

Then

$$\boxed{R_{k+1} = R_k \backslash \mathrm{Exp}(\bar{\omega} \Delta t).}$$

Calculate

$$a_k^W = R_k(a_k - b_a) + g$$

and

$$a_{k+1}^W = R_{k+1}(a_{k+1} - b_a) + g.$$

Then

$$\boxed{v_{k+1} = v_k + \frac{1}{2}(a_k^W + a_{k+1}^W)\Delta t}$$

and

$$\boxed{p_{k+1} = p_k + \frac{1}{2}(v_k + v_{k+1})\Delta t.}$$

At 200–1000 Hz this gives a much better propagation foundation.

---

5. Error dynamics

Using

$$\delta x = [\delta p, \delta v, \delta\theta, \delta b_a, \delta b_g]^T,$$

we have

$$\delta\dot{x} = F\delta x + Gn.$$

A common F structure is

$$F = \begin{bmatrix} 0 & I & 0 & 0 & 0 \\ 0 & 0 & -R[a]_\times & -R & 0 \\ 0 & 0 & -[\omega]_\times & 0 & -I \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}.$$

Noise vector:

$$n = [n_a, n_g, n_{ba}, n_{bg}]^T.$$

Corresponding G:

$$G = \begin{bmatrix} 0 & 0 & 0 & 0 \\ -R & 0 & 0 & 0 \\ 0 & -I & 0 & 0 \\ 0 & 0 & I & 0 \\ 0 & 0 & 0 & I \end{bmatrix}.$$

---

6. Proper covariance discretization

The simple approximation

$$\Phi \approx I + F\Delta t$$

is useful educationally, but I'd avoid relying solely on it in a production estimator.

Continuous covariance obeys

$$\dot{P} = FP + PF^T + GQ_cG^T.$$

We need

$$\Phi = e^{F\Delta t}$$

and

$$Q_d = \int_0^{\Delta t} e^{F\tau} G Q_c G^T e^{F^T\tau} d\tau.$$

One robust approach is Van Loan discretization.

Construct

$$\mathcal{A} = \begin{bmatrix} -F & GQ_cG^T \\ 0 & F^T \end{bmatrix} \Delta t.$$

Compute

$$e^{\mathcal{A}}.$$

Extract the appropriate blocks to recover Φ and Q_d.

Then

$$\boxed{P_{k+1} = \Phi P_k \Phi^T + Q_d.}$$

---

7. IMU calibration

A production system shouldn't assume:

$$a_{\text{calibrated}} = a_{\text{raw}} - b_a.$$

A more complete sensor model is something like

$$a_m = S_a M_a a + b_a + n_a$$

and

$$\omega_m = S_g M_g \omega + b_g + n_g.$$

Here:

- S: scale-factor errors;
- M: axis misalignment/cross-axis effects;
- b: biases.

Temperature can also matter:

$$b_a = b_a(T), \qquad b_g = b_g(T).$$

Factory calibration should remove the dominant deterministic effects before the estimator sees the measurements.

---

8. GNSS model

Do not treat GNSS as simply

$$z = p.$$

The antenna is offset from the IMU.

Let

$$l_G^E$$

be the antenna lever arm in ego coordinates.

Then:

$$\boxed{h_G(x) = p_W + R_{WE} l_G^E.}$$

Residual:

$$r_G = z_G - \left(p_W + R_{WE} l_G^E\right).$$

GNSS velocity can also be fused:

$$\boxed{z_v = v^W_{\text{antenna}} + n.}$$

For rotating vehicles, the antenna velocity contains the lever-arm rotational component:

$$v_G^W = v_E^W + R_{WE}(\omega^E \times l_G^E).$$

---

9. GNSS quality must be dynamic

Do not assign a fixed covariance such as

```
C++
R_gnss = 0.25 * I;
```

for every measurement.

Use receiver quality information:

- RTK fixed;
- RTK float;
- standalone GNSS;
- satellite geometry;
- reported covariance;
- integrity indicators;
- multipath indicators;
- correction age.

Conceptually:

$$R_{\text{RTK}-\text{fixed}} \ll R_{\text{RTK}-\text{float}} \ll R_{\text{standalone}}.$$

And bad GNSS should sometimes be rejected entirely.

---

10. Innovation consistency checking

For every measurement:

$$r = z - h(\hat{x})$$

$$S = HPH^T + R$$

and normalized innovation squared:

$$\boxed{NIS = r^T S^{-1} r.}$$

Compare against a chi-square threshold:

$$NIS < \chi^2_{m,\alpha}.$$

But production logic shouldn't merely do:

```
Plain text
if bad:
    discard
```

It should track repeated inconsistency.

For example:

```
Plain text
normal
  |
one bad innovation
  |
suspect
  |
repeated failures
  |
degraded
  |
sensor isolated
```

---

11. Wheel odometry

Wheel speed provides excellent short-term information.

For body velocity:

$$v_E = R_{EW} v_W.$$

Measurement:

$$\boxed{z_w = e_x^T R_{EW} v_W + n_w.}$$

But wheels slip.

So measurement covariance should be adaptive.

For example:

$$R_w = \begin{cases} \sigma^2_{\text{normal}} & \text{normal} \\ \sigma^2_{\text{turn}} & \text{high steering} \\ \sigma^2_{\text{slip}} & \text{slip suspected} \end{cases}$$

where

$$\sigma_{\text{slip}} \gg \sigma_{\text{normal}}.$$

---

12. Detecting wheel slip

Compare independent estimates:

$$v_{\text{wheel}}$$

against

$$v_{\text{GNSS}}$$

or inertial/LiDAR velocity.

Define:

$$e_v = \left| v_{\text{wheel}} - v_{\text{independent}} \right|.$$

Large persistent disagreement implies slip.

Also use:

- ABS/ESC signals;

- wheel-speed differences;
- steering angle;
- yaw rate;
- longitudinal acceleration.

Then inflate R_w rather than blindly trusting wheels.

---

13. Nonholonomic constraint

For normal road motion:

$$v_y^E \approx 0, \qquad v_z^E \approx 0.$$

Measurement:

$$z = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

Model:

$$h(x) = \begin{bmatrix} e_y^T R^T v \\ e_z^T R^T v \end{bmatrix}.$$

But this must be conditional.

Disable or weaken it during:

- skidding;
- high lateral acceleration;
- banked roads;
- rough terrain;
- jumps;
- unusual maneuvers.

---

14. LiDAR localization

For AVs, LiDAR-to-map localization is a major absolute/relative pose source.

First deskew the scan using the current pose trajectory:

$$\boxed{p_i^{E_r} = T_{WE}(t_r)^{-1} T_{WE}(t_i) T_{EL} p_i^L.}$$

Then register the deskewed cloud against the map.

---

15. Point-to-plane map residual

Suppose transformed LiDAR point is

$$p_i^W = T_{WE} T_{EL} p_i^L.$$

Map correspondence is q_i with normal n_i.

Residual:

$$\boxed{r_i = n_i^T (p_i^W - q_i).}$$

Minimize:

$$\boxed{T^* = \arg\min_T \sum_i \rho\left(\frac{r_i^2}{\sigma_i^2}\right).}$$

Use robust loss ρ because some points belong to:

- moving cars;
- pedestrians;
- vegetation;
- changed map structures;
- bad correspondences.

---

16. Dynamic-object filtering before map matching

This is particularly important given our previous object-motion discussion.

Do not use obvious moving-object points for static-map localization.

Conceptually:

```
Plain text
```

```
LiDAR
  |
deskew
  |
semantic/dynamic mask
  |
  +---- car -------- X
  +---- pedestrian - X
  +---- cyclist ---- X
  |
static points
  |
map registration
```

Otherwise a large truck beside the ego vehicle can bias localization.

---

17. LiDAR registration degeneracy

Suppose you drive through a long tunnel.

LiDAR may constrain:

- lateral position well;
- roll/pitch well;

but longitudinal position poorly.

The registration Hessian

$$H = J^T W J$$

reveals this.

Eigen-decompose:

$$H = V \Lambda V^T.$$

If

$$\lambda_i \ll \epsilon,$$

direction v_i is weakly constrained.

Instead of pretending the full LiDAR pose is equally accurate, increase uncertainty along that direction.

This is critical for AV-grade localization.

---

18. Don't reduce every LiDAR update to a fixed 6-DOF covariance

A useful map matcher should return:

$$\boxed{\{T_{\text{lidar}}, \Sigma_{\text{lidar}}, \text{fitness}, \text{inlier ratio}, \text{degeneracy}\}}$$

rather than merely

```
C++

Pose lidarPose;
```

The localization system then knows whether to trust it.

---

19. Camera localization

Camera information can enter through landmarks/features.

For world landmark P_i^W:

$$P_i^C = T_{CE} T_{EW} P_i^W.$$

Projection:

$$\hat{z}_i = \pi(P_i^C).$$

Residual:

$$\boxed{r_i = z_i - \pi(T_{CE} T_{EW} P_i^W).}$$

Minimize robust reprojection error.

Camera localization becomes especially useful where LiDAR geometry is weak but visual landmarks are distinctive.

---

20. Fixed-lag smoother

This is where I would make the biggest architectural change from the previous C++ example.

Maintain recent states:

$$X_{k-N}, \dots, X_k.$$

For example, a several-second window.

Solve:

$$\boxed{X^* = \arg\min_X \left[\|r_{\text{prior}}\|^2 + \sum r_{\text{IMU}}^2 + \sum r_{\text{GNSS}}^2 + \sum r_{\text{wheel}}^2 + \sum r_{\text{LiDAR}}^2 + \sum r_{\text{camera}}^2 \right]}$$

Each residual is covariance weighted.

---

21. IMU preintegration

Suppose keyframes are separated by 100 ms but IMU operates at 400 Hz.

There are about 40 IMU samples between keyframes.

Instead of creating 40 separate graph states, preintegrate them.

Compute

$$\Delta R_{ij}, \qquad \Delta v_{ij}, \qquad \Delta p_{ij}.$$

The rotation residual becomes approximately

$$\boxed{r_R = \backslash\text{Log}\left[\Delta \tilde{R}_{ij}^T R_i^T R_j\right].}$$

Velocity residual:

$$\boxed{r_v = R_i^T (v_j - v_i - g\Delta t) - \Delta \tilde{v}_{ij}.}$$

Position residual:

$$\boxed{r_p = R_i^T \left[p_j - p_i - v_i \Delta t - \frac{1}{2} g \Delta t^2 \right] - \Delta \tilde{p}_{ij}.}$$

Plus bias evolution residuals.

---

22. Bias correction of preintegration

Preintegrated IMU terms depend on assumed biases.

Rather than recomputing everything for tiny bias changes:

$$\Delta R(b_g + \delta b_g) \approx \Delta \bar{R} \backslash \mathrm{Exp}(J_{R,b_g} \delta b_g)$$

$$\Delta v \approx \Delta \bar{v} + J_{v,b_a} \delta b_a + J_{v,b_g} \delta b_g$$

$$\Delta p \approx \Delta \bar{p} + J_{p,b_a} \delta b_a + J_{p,b_g} \delta b_g.$$

This makes nonlinear optimization much more efficient.

---

23. Delayed measurements

This is one of the biggest differences between demo code and real systems.

Suppose:

Plain text

```
IMU sample:
timestamp = 12.000 s
arrives    = 12.001 s

GNSS:
timestamp = 11.850 s
arrives    = 12.020 s

LiDAR registration:
scan timestamp = 11.900 s
result arrives = 12.060 s
```

You cannot simply apply the LiDAR result to the current 12.060 s state.

It belongs to the past.

---

24. Out-of-sequence measurement handling

Maintain a timestamped state/measurement buffer:

```
Plain text

11.80    11.85    11.90    11.95    12.00    12.05
 |         |        |        |        |        |
state     GNSS    LiDAR              IMU      now
```

When the LiDAR result for 11.90 arrives:

1. insert/update the constraint at 11.90;
2. optimize the smoother;
3. obtain corrected historical trajectory;
4. propagate the correction forward to the present.

This is a major reason a fixed-lag smoother is attractive.

---

25. Time synchronization

Every measurement must have:

$$\boxed{t_{\text{measurement}}}$$

not merely its arrival time.

Maintain known time offsets:

$$t_{\text{corrected}} = t_{\text{sensor}} + \Delta t_{\text{sensor}}.$$

For LiDAR deskewing, per-point timing matters:

$$t_i = t_{\text{scan}} + dt_i.$$

At 30 m/s, a 10 ms timing error gives roughly

$$30(0.01) = 0.3m$$

of translational error before considering rotation.

26. Pose buffer

The localization service should expose something like:

```cpp
PoseQueryResult getPose(double timestamp);
```

returning:

```cpp
struct PoseQueryResult {
    double timestamp;

    Eigen::Vector3d position;
    Eigen::Quaterniond orientation;
    Eigen::Vector3d velocity;

    Eigen::Matrix<double, 6, 6> poseCovariance;

    LocalizationQuality quality;
};
```

This becomes the central interface for perception.

---

27. Pose interpolation

For two neighboring estimates:

$$T(t_a), T(t_b),$$

translation can be interpolated:

$$p(t) = (1-\alpha)p_a + \alpha p_b$$

with

$$\alpha = \frac{t - t_a}{t_b - t_a}.$$

Orientation:

$$\boxed{q(t) = SLERP(q_a, q_b, \alpha).}$$

For high-accuracy deskewing, an IMU-based continuous-time interpolation/propagation model is preferable to simply interpolating sparse localization outputs.

---

28. Localization integrity state

A production localization system needs an explicit state machine.

For example:

```cpp
enum class LocalizationMode
{
    INITIALIZING,
    NOMINAL,
    GNSS_DEGRADED,
    LIDAR_DEGRADED,
    DEAD_RECKONING,
    RECOVERING,
    INVALID
};
```

This matters just as much as the numerical pose.

The planner needs to know whether

$$\sigma_{xy} = 0.05m$$

or

$$\sigma_{xy} = 3m.$$

---

29. Health metrics

I'd continuously monitor at least:

$$\sigma_x, \sigma_y, \sigma_z$$

$$\sigma_{\text{roll}}, \sigma_{\text{pitch}}, \sigma_{\text{yaw}}$$

plus:

- GNSS NIS;
- LiDAR registration residual;
- LiDAR degeneracy;
- visual inlier ratio;
- IMU saturation;
- bias magnitude;
- wheel-slip score;
- sensor timestamp age;
- time-offset health;
- map confidence.

---

30. Sensor fault isolation

Suppose GNSS jumps 15 m.

Don't immediately move the vehicle estimate.

Compute

$$NIS_{\text{GNSS}} = r_G^T S_G^{-1} r_G.$$

Compare it against the expected distribution.

Also compare GNSS against independent sources:

$$T_{\text{GNSS}} \quad \text{vs} \quad T_{\text{LiDAR}} \quad \text{vs} \quad T_{\text{inertial}}.$$

If GNSS alone disagrees:

$$\boxed{\text{deweight/isolate GNSS}}$$

rather than corrupting localization.

---

31. Cross-sensor consistency

You can form independent residuals such as

$$r_{GL} = \operatorname{Log}(T_{\text{GNSS}}^{-1} T_{\text{LiDAR}})$$

and

$$r_{LI} = \operatorname{Log}(T_{\text{LiDAR}}^{-1} T_{\text{inertial}}).$$

Persistent disagreement provides evidence of a sensor fault, bad calibration, bad map match or time-sync problem.

---

32. Initialization

Initialization itself should be a state machine.

Stage 1 — stationary IMU initialization

Estimate gyro bias:

$$\hat{b}_g = \frac{1}{N} \sum_k \omega_{m,k}.$$

Use gravity to estimate roll/pitch.

Stage 2 — position

Use high-quality GNSS/map localization.

Stage 3 — yaw

Use one or more of:

- GNSS heading;
- dual-antenna GNSS;
- LiDAR map alignment;
- vehicle motion;
- visual map alignment.

Stage 4 — velocity

Use GNSS velocity/wheels.

Stage 5 — validation

Only declare `NOMINAL` after covariance and cross-sensor consistency pass thresholds.

---

33. Relocalization

Suppose the system loses map localization in a tunnel.

Continue:

$$\text{IMU} + \text{wheel dead reckoning}.$$

Covariance increases.

When map matching becomes available again, don't blindly snap to the first candidate.

Generate candidates:

$$T_1, T_2, \ldots, T_n.$$

Check each against:

- predicted state;
- covariance;
- GNSS if available;
- map geometry;
- registration score.

Accept only a statistically/geometrically consistent solution.

---

34. Map-frame correction must not destabilize control

Suppose relocalization changes global position by 1.5 m.

A direct instantaneous jump in the control frame can be undesirable.

A common architecture separates:

$$\boxed{\text{smooth local odometry frame}}$$

from

$$\boxed{\text{globally corrected map frame}}.$$

Think of:

$$T_{WO}$$

as map/world → local odometry correction and

$$T_{OE}$$

as smooth ego odometry.

Then:

$$\boxed{T_{WE} = T_{WO} T_{OE}.}$$

The local frame remains smooth for control while the global transform absorbs map corrections.

---

35. This also helps perception

For local tracking and planning, use a smooth local frame.

For:

- HD-map association;
- route localization;
- long-term world tracks;

use the global/map frame.

That avoids injecting map-localization discontinuities into local object motion.

---

36. Recommended C++ module structure

I would no longer put everything in one `pose_estimator.cpp`.

Use something like:

Plain text

```
localization/
|
+-- include/
|   |
|   +-- state.hpp
|   +-- se3.hpp
|   +-- imu_types.hpp
```

```
|   +-- sensor_measurements.hpp
|   +-- calibration.hpp
|   +-- pose_buffer.hpp
|   +-- localization_health.hpp
|
+-- imu/
|   |
|   +-- imu_calibration.cpp
|   +-- imu_propagator.cpp
|   +-- imu_preintegration.cpp
|
+-- eskf/
|   |
```

---

37. Main processing loop

At the architectural level:

C++

```cpp
void LocalizationSystem::processImu(const Imu& imu)
{
    Imu calibrated = imuCalibration_.correct(imu);

    eskf_.propagate(calibrated);

    poseBuffer_.insert(
        eskf_.timestamp(),
        eskf_.state(),
        eskf_.covariance());

    smoother_.addImu(calibrated);

    health_.updateImu(calibrated);
}
```

GNSS:

C++

```cpp
void LocalizationSystem::processGnss(const Gnss& gnss)
{
    if (!gnssIntegrity_.validate(gnss))
        return;

    smoother_.addGnssFactor(gnss);

    if (isApproximatelyCurrent(gnss.timestamp))
```

```
        eskf_.updateGnss(gnss);
}
```

LiDAR:

```
    C++

void LocalizationSystem::processLidar(LidarScan scan)
{
    deskewer_.deskew(
        scan,
        poseBuffer_);

    RegistrationResult result =
        mapMatcher_.registerScan(scan);

    if (!result.valid)
        return;

    if (result.degenerate)
        result.covariance =
            degeneracyHandler_.inflate(
                result.covariance,
                result.informationMatrix);

    smoother_.addLidarFactor(result);
}
```

Optimization:

```
    C++

void LocalizationSystem::optimize()
{
    OptimizedTrajectory trajectory =
        smoother_.optimize();

    State corrected =
        trajectory.latestState();

    eskf_.applySmoothCorrection(corrected);

    poseBuffer_.replaceRecentTrajectory(
        trajectory);

    health_.update(trajectory);
}
```

38. The nonlinear objective

The back-end ultimately solves something resembling

$$
\begin{aligned}
X^* = \arg\min_X \; & \|r_{\text{prior}}\|^2_{\Sigma_P^{-1}} \\
& + \sum_k \|r_{\text{IMU,k}}\|^2_{\Sigma_{I,k}^{-1}} \\
& + \sum_j \rho_G\left(\|r_{\text{GNSS,j}}\|^2_{\Sigma_{G,j}^{-1}}\right) \\
& + \sum_l \rho_L\left(\|r_{\text{LiDAR,l}}\|^2_{\Sigma_{L,l}^{-1}}\right) \\
& + \sum_w \rho_W\left(\|r_{\text{wheel,w}}\|^2_{\Sigma_{W,w}^{-1}}\right) \\
& + \sum_c \rho_C\left(\|r_{\text{camera,c}}\|^2_{\Sigma_{C,c}^{-1}}\right).
\end{aligned}
$$

That is much closer to what I'd call an AV-grade mathematical formulation than a single sequential EKF.

---

39. Failure scenarios the design must survive

A serious system should explicitly test scenarios such as:

Scenario	Expected behavior
RTK GNSS available	cm-level global correction
Urban canyon	reject/deweight multipath GNSS
GNSS outage	LiDAR + IMU + wheels
Tunnel	inertial/wheel/LiDAR dead reckoning
Featureless tunnel	detect LiDAR degeneracy
Wheel slip	inflate wheel covariance
LiDAR obscured	camera/GNSS/IMU continue
Camera glare	reject poor visual constraints

Scenario	Expected behavior
IMU saturation	flag degraded propagation
Old LiDAR result	update historical smoother state
Map mismatch	reject registration
Localization recovery	statistically validated relocalization
Moving traffic	remove dynamic LiDAR points from map matching
Timestamp fault	detect cross-sensor inconsistency

---

40. Connection to our camera + LiDAR perception stack

This gives us a much cleaner full AV architecture:

Plain text

```
                  LOCALIZATION
                       |
           T_WE(t), v(t), P(t)
                       |
         +-------------+-------------+
         |                           |
         v                           v
    LiDAR deskew                Camera poses
         |
         v
dynamic-object removal
         |
         v
   map localization
         |
         +------ localization correction
         |
         v
   deskewed LiDAR
         |
```

There is an important separation here:

$$\boxed{T_{WE}(t) = \text{ego pose from localization}}$$

while

$$\boxed{T_{WO_j}(t) = \text{dynamic-object pose from tracking.}}$$

Together they give the transformation we derived earlier:

$$\boxed{p_i^{E_r} = T_{WE}(t_r)^{-1} T_{WO}(t_r) T_{WO}(t_i)^{-1} T_{WE}(t_i) T_{EL} p_i^L.}$$

So at this point the major components we've discussed—**pose estimation, LiDAR deskewing, dynamic-object compensation, camera/LiDAR fusion, JPDA/MHT tracking and world-frame tracking—fit into one consistent mathematical architecture**.

For an implementation, I would build the next version as a **multi-file C++17 project using Eigen**, first implementing the production-style `SO3/SE3`, midpoint IMU propagator, 15-state ESKF, timestamped pose buffer and LiDAR deskewer; then add GNSS/wheel updates and LiDAR point-to-plane localization; finally put a fixed-lag factor-graph smoother behind it. That sequencing keeps each layer testable before adding the next.